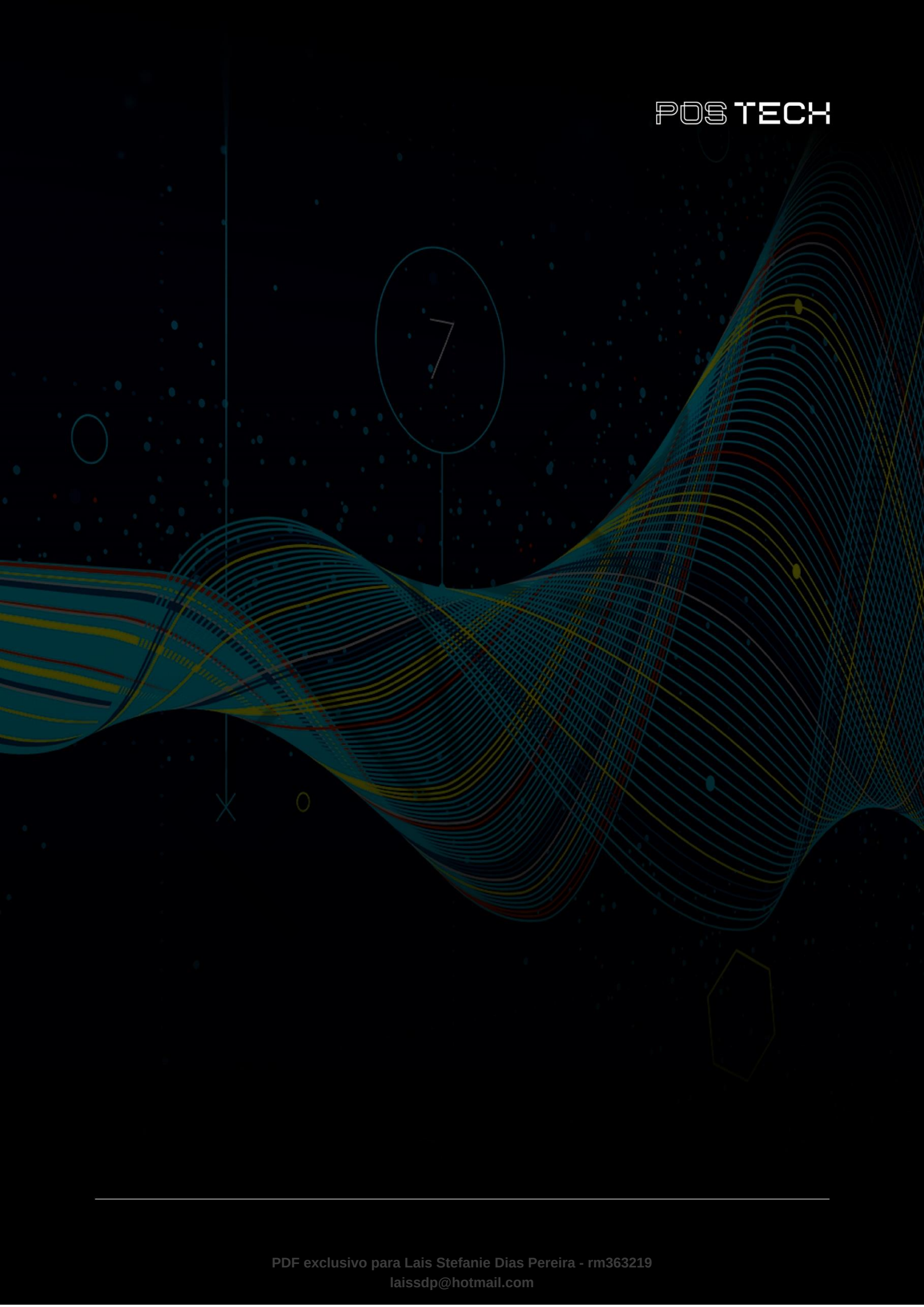




SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
MERCADO, CASES E TENDÊNCIAS	38
O QUE VOCÊ VIU NESTA AULA?	39
REFERÊNCIAS.....	40

O QUE VEM POR AÍ?

Agora teremos contato com **containerização** utilizando Docker.

A **containerização** é uma prática essencial no desenvolvimento de software, pois permite empacotar a aplicação e todas as suas dependências em um ambiente isolado e portátil chamado de **container**. Isso garante que a aplicação funcione de maneira consistente em diferentes ambientes, desde o desenvolvimento até a produção, eliminando problemas comuns relacionados à configuração e dependências da aplicação.

O termo container é usado porque, assim como um container de transporte, que é útil para padronizar o transporte de coisas, ele encapsula todos os elementos necessários para rodar uma aplicação de forma segura e eficiente, independentemente do sistema onde será executado (Windows, Linux etc).

Com o **Docker** é possível desenvolver, implantar e escalar aplicações de maneira rápida e confiável em containers, promovendo uma maior flexibilidade de gerenciamento ao longo do ciclo de vida de uma aplicação.

HANDS ON

Nesta aula, serão abordados os conceitos fundamentais de virtualização e containerização, com foco na tecnologia Docker. Será explicado o que são máquinas virtuais (VMs) e containers, suas diferenças, vantagens e aplicações práticas. A aula também cobrirá a instalação e configuração do Docker, a criação de imagens Docker a partir de Dockerfiles, a execução de containers e o gerenciamento de volumes para persistência de dados.

Além disso, será introduzido o Docker Compose, uma ferramenta para orquestrar aplicações multi-container, com exemplos práticos de como criar e gerenciar ambientes complexos. Ao final, os alunos e alunas terão a oportunidade de aplicar esses conceitos em exercícios práticos, criando e executando containers Docker para diferentes cenários, incluindo a execução de scripts Python e a configuração de um banco de dados MySQL com persistência de dados.

SAIBA MAIS

Como mencionado, a presente fase do curso aborda conceitos fundamentais de containerização com Docker, o qual é uma tecnologia amplamente utilizada no desenvolvimento de software. Para compreensão dessa tecnologia, precisamos entender o que é virtualização e como ela é importante na containerização. Ambos são métodos importantes para otimizar recursos e permitir um gerenciamento mais eficiente da infraestrutura de tecnologia. A seguir, definiremos e compararemos os conceitos de virtualização e containerização, além de abordar como essas tecnologias estão relacionadas ao Docker.

Introdução à Virtualização

Máquinas físicas e suas limitações

Os computadores que utilizamos no dia a dia, como laptops e smartphones, são exemplos de máquinas físicas. No mundo da TI, máquinas físicas (como servidores) desempenham um papel crítico em atividades como hospedagem de websites, processamento de grandes volumes de dados, e treinamento de modelos de Machine Learning ou de Inteligência Artificial. No entanto, essas máquinas apresentam diversas limitações:

- **Alto Custo:** a manutenção de máquinas físicas exige investimentos significativos em hardware, o que pode ser caro, especialmente ao escalar para atender demandas crescentes e testes rápidos para POCs (prova de conceito).
- **Escalabilidade Limitada:** para adicionar mais capacidade, é necessário adquirir hardware adicional, o que pode levar tempo e gerar altos custos operacionais.
- **Manutenção Complexa:** a manutenção de hardware físico envolve custos com peças, atualizações, e o risco de falhas físicas que podem impactar a entrega de soluções de software e de dados.

Essas limitações criam a necessidade de soluções mais eficientes e flexíveis, como a **virtualização**, que permite otimizar o uso dos recursos físicos de forma mais prática.

Máquinas virtuais

As máquinas virtuais (VMs) são uma solução eficaz para superar as limitações das máquinas físicas. Uma VM é um ambiente de computação simulado, que se comporta como um computador físico, mas é executado como um software em uma máquina host (física). Isso permite a criação de várias VMs em uma única máquina, cada uma operando de forma isolada e independente.

Na década de 1960, a virtualização surgiu nos mainframes da IBM com o software VM CMS, permitindo a criação de máquinas virtuais. Em 1999, a VMware trouxe a virtualização para a plataforma x86, aumentando a eficiência do processamento. E a partir de 2005, fabricantes como Intel e AMD implementaram essa tecnologia diretamente no hardware, melhorando o desempenho das aplicações.

Definição e aplicação da virtualização

Virtualização é o processo de criar uma versão virtual de um recurso de computação, que pode incluir desde a virtualização completa de um sistema (incluindo hardware e software) até a virtualização de componentes específicos, como armazenamento. Virtualizar sistemas inteiros resulta em uma VM, proporcionando escalabilidade e otimização de recursos de forma mais prática e econômica do que a ampliação de máquinas físicas.

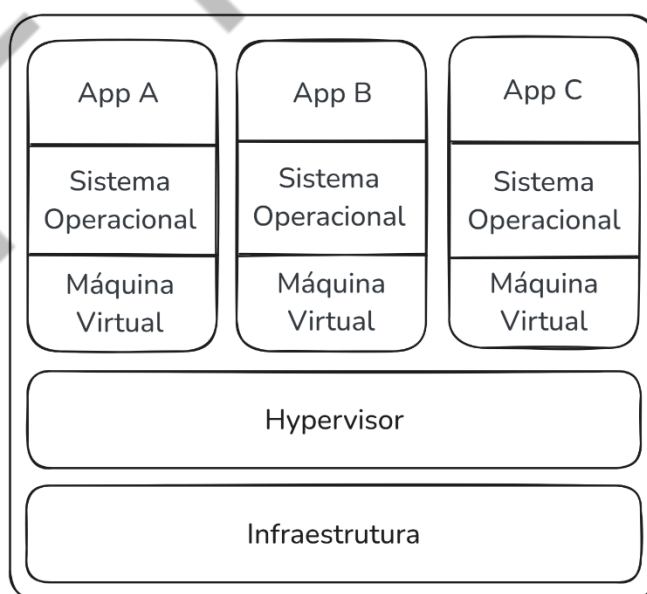


Figura 1 - Arquitetura de virtualização baseada em máquinas virtuais

Fonte: Adaptado de [Docker](#) (2025)

A Figura 1 representa uma arquitetura de virtualização baseada em máquinas virtuais (VMs). A seguir, está o que cada camada dessa arquitetura indica:

Infraestrutura: a camada física, composta pelo hardware do servidor (CPU, memória, armazenamento etc.).

Hypervisor: um software que cria e gerencia as VMs, permitindo que vários sistemas operacionais compartilhem o mesmo hardware físico. Existem dois tipos de hypervisores: do tipo 1 (executados diretamente sobre o hardware); tipo 2 (executados sobre um sistema operacional).

Máquina Virtual: cada VM simula um computador completo, com seus próprios recursos de CPU, memória e armazenamento. Cada uma executa um sistema operacional separado.

Sistema Operacional: cada VM possui seu próprio sistema operacional, que é isolado das outras VMs.

Aplicações (App A, App B, App C): executadas em cada VM, são as aplicações específicas que os usuários utilizam.

Essa abordagem oferece isolamento entre aplicações, já que cada uma é executada em seu próprio ambiente virtualizado, o que pode demandar mais recursos por conta da duplicação dos sistemas operacionais em cada VM.

Containerização

A virtualização pode ocorrer em diferentes níveis, além de virtualizar um computador inteiro, é possível virtualizar componentes específicos do sistema. Uma forma de virtualização que tem destaque no mercado é a virtualização em nível de sistema operacional (OS-level), conhecida como “containerização”. Nesse contexto, o “ambiente” refere-se ao conjunto de recursos necessários para executar uma aplicação, incluindo sistema operacional, bibliotecas, configurações e outras dependências.

Virtualização em OS-level

Na virtualização em OS-level ou containerização, o objetivo é fornecer ambientes isolados para a execução de aplicações, sem replicar o hardware ou o kernel do sistema operacional. Diferente das máquinas virtuais tradicionais, onde

cada VM executa seu próprio kernel, os containers compartilham o kernel do sistema host, mas mantêm isolamento total em termos de bibliotecas e dependências.

Cada container é executado em um espaço de usuário separado, utilizando o kernel do host para comunicação com o hardware e gerenciamento de processos. Isso permite que os containers sejam leves, iniciem rapidamente e utilizem os recursos de forma eficiente, pois não há a sobrecarga de emular o hardware completo ou executar múltiplos kernels.

Containers e suas características

Containers são ambientes isolados que contêm todos os componentes necessários para uma aplicação funcionar, como o código-fonte, bibliotecas e configurações específicas. Eles garantem que uma aplicação seja executada de forma consistente, independentemente do ambiente subjacente. Essa abordagem oferece vários benefícios:

- **Isolamento:** cada container opera de forma independente dos outros, o que significa que problemas em um container não afetam os demais. Isso é especialmente útil em cenários de **microsserviços**, onde diferentes partes de uma aplicação são executadas em containers separados.
- **Portabilidade e reprodutibilidade:** como os containers encapsulam tudo o que uma aplicação necessita, é possível movê-los entre diferentes ambientes (desenvolvimento, teste/homologação e produção) sem que haja variações no comportamento da aplicação. Essa consistência é fundamental para o DevOps e a entrega contínua.
- **Desempenho e eficiência:** compartilhando o kernel do sistema host, os containers consomem menos recursos em comparação com máquinas virtuais, resultando em tempos de inicialização muito rápidos. Isso os torna ideais para aplicações que exigem alta disponibilidade e escalabilidade dinâmica.
- **Facilidade de gestão de dependências:** cada container inclui suas próprias bibliotecas e versões de dependências, eliminando conflitos com outras aplicações que possam rodar no mesmo sistema.

Qual a definição de containerização?

A containerização pode ser vista por duas perspectivas: primeiro, como uma forma de virtualização no OS-level; segundo, como o processo de agrupar uma aplicação e todas as suas dependências em um único ambiente para garantir sua portabilidade e consistência.

VMs X Containers

Containers fornecem ambientes leves que isolam aplicações e suas dependências, sendo ideais para portabilidade e microsserviços. VMs replicam sistemas completos, incluindo o sistema operacional, e são adequadas para aplicações monolíticas e ambientes isolados. Containers são mais ágeis e ocupam menos espaço, enquanto VMs são mais robustas, oferecendo isolamento completo. Cada tecnologia possui vantagens dependendo do cenário de uso e das necessidades de desenvolvimento.

<i>VMs</i>	<i>Container</i>
• Isolamento completo; • Executa múltiplos sistemas operacionais; • Em caso de utilização de aplicações monolíticas que exigem mais recursos dedicados.	• Agilidade, portabilidade e eficiência de recursos; • Para aplicações que utilizam arquitetura de microsserviços ou que necessitam de rápida escalabilidade; • Ideais para ambientes de desenvolvimento que valorizam a flexibilidade e a capacidade de executar várias instâncias em menos tempo e com menos uso de recursos.

Tabela 1 – VMs x Container
Fonte: Elaborado pelo autor (2025)

Containerização com Docker

Como já foi apresentado, os containers fornecem um ambiente isolado e leve para a execução de aplicações. Eles encapsulam tudo que é necessário para a

aplicação rodar de forma consistente em diferentes ambientes, como código-fonte, bibliotecas, dependências e configurações do sistema.

Docker

Docker é uma plataforma que utiliza a virtualização em nível de sistema operacional (OS-level) para entregar aplicações em pacotes padronizados chamados containers. Por ser uma ferramenta de código aberto, qualquer pessoa pode utilizá-lo, modificá-lo e distribuí-lo. De acordo com uma [pesquisa](#) do Stack Overflow, Docker é atualmente uma das tecnologias mais utilizadas e preferidas por pessoas desenvolvedoras. O principal diferencial do Docker é fornecer ferramentas robustas para construir, distribuir e executar containers, além de gerenciar o ciclo de vida completo dos containers, desde o desenvolvimento local até a execução em ambientes de produção.

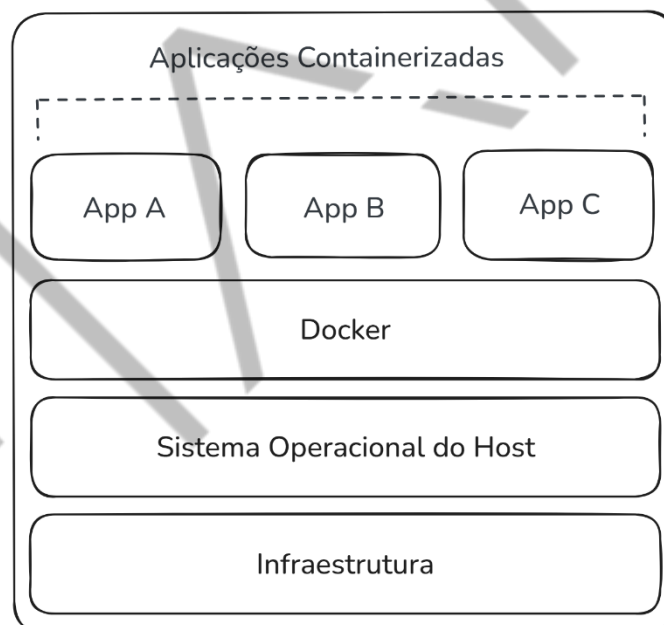


Figura 2 - Arquitetura baseada em containers
Fonte: Adaptado de [Docker](#) (2025)

A Figura 2 representa uma arquitetura baseada em containers, com o uso do Docker para gerenciar as aplicações. A seguir estão as explicações das camadas representadas:

Infraestrutura: o hardware físico do servidor, que fornece os recursos de CPU, memória e armazenamento.

Sistema Operacional do Host: o sistema operacional principal que roda diretamente sobre o hardware.

Docker: camada de virtualização que gerencia os containers. Diferente das VMs tradicionais, o Docker compartilha o sistema operacional do host e fornece um ambiente isolado para cada aplicação.

Aplicações Containerizadas (App A, App B, App C): são as aplicações que rodam em containers. Cada container compartilha o mesmo kernel do sistema operacional do host, mas é isolado para garantir que a execução de uma aplicação não afete as outras.

Essa abordagem permite que as aplicações sejam executadas de forma mais eficiente em termos de recursos, já que evita a sobrecarga de múltiplos sistemas operacionais. Além disso, facilita a portabilidade, uma vez que os containers podem ser facilmente movidos entre diferentes ambientes.

O Docker tem uma grande base de usuários e é uma das ferramentas mais utilizadas atualmente. O que o Docker oferece como plataforma de software? Ferramentas para construir, distribuir e executar containers, assim como gerenciamento do ciclo de vida dos containers, do desenvolvimento até a produção.

Docker Desktop

Para começar a utilizar o Docker, você precisará instalar o [Docker Desktop](#). O Docker Desktop é uma aplicação de desktop disponível para Mac, Linux e Windows, o qual fornece uma interface gráfica simples (também chamada de GUI) para gerenciar seus containers, aplicações e imagens.

Arquitetura cliente-servidor do Docker Engine

O Docker Desktop também inclui o **Docker Engine**, que segue uma arquitetura de cliente-servidor. Nesta arquitetura, o cliente Docker, geralmente acessado via CLI, envia comandos ao Docker Daemon (servidor), o serviço principal que constrói (faz o build), executa e gerencia os objetos Docker. O Docker Daemon precisa estar em execução para que as operações do Docker sejam realizadas, enquanto o cliente pode ser usado para executar comandos de forma interativa, seja por meio da CLI ou de uma GUI.

Visão geral dos objetos do Docker

Os principais objetos no Docker são imagens, containers e volumes.

- **Imagens:** uma **imagem** é uma camada imutável que serve como modelo para a criação de containers. Para criar uma imagem, você escreve um arquivo de configuração chamado **Dockerfile**, que define todas as instruções necessárias para configurar o ambiente, instalar pacotes e copiar arquivos.
- **Containers:** um container Docker é uma instância em execução de uma imagem. É um ambiente isolado, criado com base nas especificações definidas na imagem. Você pode executar, parar, reiniciar e destruir containers sem afetar o sistema operacional subjacente.
- **Volumes:** volumes do Docker são usados para persistência de dados, permitindo que os dados sejam mantidos mesmo que o container seja removido. São gerenciados pelo Docker e armazenados fora do sistema de arquivos do contêiner.

O processo para configurar um ambiente utilizando Docker geralmente segue as seguintes etapas:

- **Escrever um Dockerfile:** um **Dockerfile** especifica uma sequência de instruções, como a escolha de uma imagem base, instalação de dependências e cópia de arquivos necessários para a aplicação.
- **Fazer o build da imagem Docker:** a partir do **Dockerfile**, o comando **docker build** é usado para criar uma imagem. Essa imagem serve como o "modelo" do ambiente que será executado.
- **Executar um container Docker:** com a imagem pronta, o comando **docker run** é utilizado para criar e iniciar um container, que rodará a aplicação em um ambiente isolado.

Para facilitar o entendimento, podemos comparar o uso do Docker com a construção de casas pré-fabricadas:

- Onde o Dockerfile é como o projeto de uma casa, contendo todas as instruções e especificações para a construção;

- A imagem Docker é o modelo pré-fabricado da casa, criado com base no projeto;
- E o container Docker é a casa construída, pronta para ser utilizada. Cada container é uma instância única, mas pode ser recriada a partir do mesmo modelo (imagem Docker) quantas vezes for necessário.

O Docker Daemon gerencia objetos Docker, como imagens Docker e containers Docker. Criar imagens Docker e executar containers Docker requer certas ações em uma ordem específica, quais são essas ações?

- 1) Escrever um Dockerfile;
- 2) Fazer o build de uma imagem Docker;
- 3) Executar um container do Docker.

Registro de imagens e containers Docker

Os registros Docker são locais centralizados para armazenar e gerenciar as imagens Docker. Basicamente, é como um repositório on-line que facilita o armazenamento e a troca de imagens entre diferentes usuários e ambientes. Ele permite que você salve suas imagens e as disponibilize para outros sistemas, para que possam ser utilizadas para criar containers.

Os registros são divididos em **repositórios**, que são coleções de imagens. Cada imagem em um repositório é identificada por uma **tag**, que serve para diferenciar versões diferentes da mesma imagem.

Por exemplo, você pode ter um repositório para uma aplicação chamada "meu_app", e dentro desse repositório, pode ter uma imagem com tag "v1.0" e outra com tag "v2.0". Dessa forma, você consegue manter várias versões da mesma aplicação em um único local organizado.

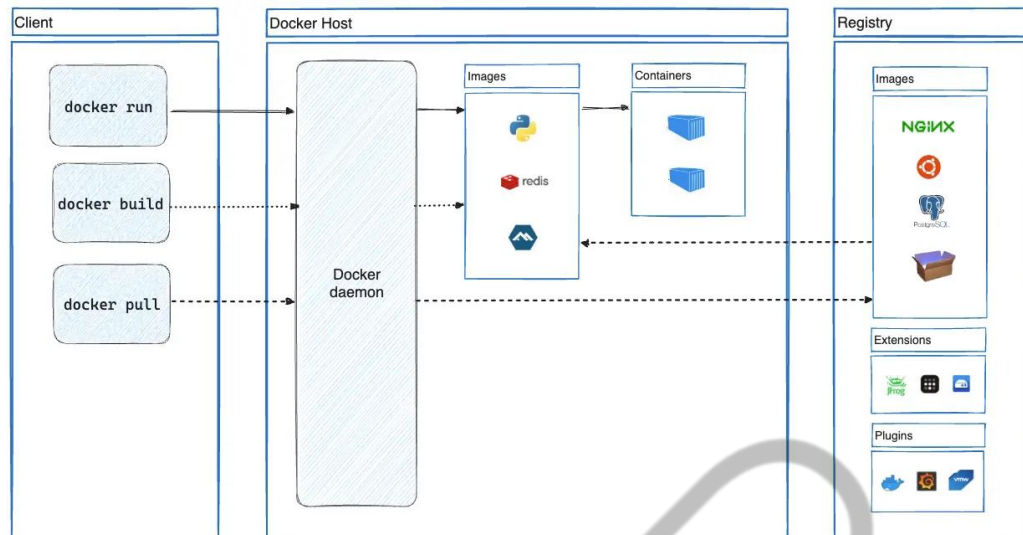


Figura 3 - Arquitetura básica do Docker.

Fonte: [Docker \(2025\)](#)

A Figura 3 apresenta a arquitetura básica do Docker, que é composta por três partes principais: o **Cliente**, o **Docker Host** e o **Registry** (ou registro Docker, como tratamos até aqui).

O Cliente é a interface onde o usuário interage com o Docker, utilizando comandos como `docker build`, `docker run` e `docker pull` (nas seções a seguir, os comandos do Docker serão melhor explicados!). Onde:

- `docker build`: constrói uma imagem a partir de um arquivo Dockerfile.
- `docker run`: executa um container Docker a partir de uma imagem.
- `docker pull`: baixa uma imagem do Registry para o Docker Host.

O Docker Host é onde os containers são executados e gerenciados. Contém o Docker daemon, que é responsável por gerenciar as imagens e os containers. Onde:

- **Imagens:** são templates prontos para a criação de containers. Exemplos incluem Python, Redis e outras aplicações.
- **Containers:** são instâncias em execução das imagens, que funcionam de forma isolada e utilizam os recursos fornecidos pela imagem.
- **Registry:** é um repositório onde as imagens Docker são armazenadas. O Registry pode ser público (como o Docker Hub) ou privado, onde:

- **Imagens:** exibe exemplos de imagens disponíveis no Registry, como NGINX, Ubuntu e PostgreSQL.
- **Extensões e Plugins:** fornecem funcionalidades adicionais ao Docker, como integração com ferramentas de terceiros (JFrog, Sonatype Nexus etc.) e monitoramento (Prometheus, Grafana etc.).

As setas na Figura 3 indicam como os comandos interagem entre o Cliente, Docker Host e Registry, mostrando o fluxo de criação, execução e download de imagens e containers. Existem dois tipos principais de registros: registro público e registro privado.

Registro público

O exemplo mais conhecido de registro público é o [Docker Hub](#). O Docker Hub é o registro público padrão, oferecido pela Docker Inc., onde você pode encontrar e compartilhar milhões de imagens de containers de forma gratuita ou com planos pagos para recursos mais avançados.

No Docker Hub é possível encontrar imagens oficiais, que são imagens Docker verificadas, como de sistemas operacionais (Ubuntu, Alpine) e ferramentas populares como Python, MySQL, Nginx e Node.js, por exemplo.

Registro privado

Em muitos casos, empresas preferem usar registros privados para armazenar suas imagens, especialmente se houver dados sensíveis envolvidos. Você pode configurar seu próprio Docker Registry privado (utilizando a ferramenta Docker Registry oficial, que também é open-source) ou utilizar serviços como Amazon Elastic Container Registry (ECR), Azure Container Registry (ACR) ou Google Artifact Registry.

Fluxo de trabalho com registros Docker

Quando você trabalha com Docker, o fluxo de trabalho envolvendo um registro é:

Build da Imagem: você cria uma imagem Docker a partir de um Dockerfile que contém as instruções para criar o ambiente necessário para sua aplicação.

Push da Imagem: depois de criar a imagem, você realiza o “push” dessa imagem para um registro como o Docker Hub. Essa ação envia a imagem para o repositório do registro, onde ela fica armazenada e pode ser compartilhada.

Pull da Imagem: outros desenvolvedores ou servidores podem “realizar o pull” dessa imagem para utilizá-la, puxando-a diretamente do registro.

Vantagens de registros Docker

A utilização de registros Docker facilita o armazenamento centralizado e o compartilhamento de imagens. Isso é especialmente útil quando você está colaborando com uma equipe ou distribuindo sua aplicação para diferentes ambientes, como desenvolvimento, teste/homologação e produção.

O uso de tags em repositórios Docker permite gerenciar várias versões de uma mesma aplicação, garantindo que versões específicas sejam sempre acessíveis e facilmente identificáveis.

E, ainda, os registros Docker são fundamentais para pipelines de CI/CD, pois permitem que imagens sejam automaticamente construídas, armazenadas e usadas para criar containers em diferentes ambientes. Por exemplo, quando uma nova versão do código é criada, um pipeline pode ser configurado para construir uma nova imagem, fazer o push para um registro e, em seguida, implantar containers atualizados automaticamente.

Quais são os principais benefícios da utilização de registros Docker? São a centralização, compartilhamento e controle de versões de imagens Docker.

Leitura de Dockerfiles e Execução de Containers - Recapitulando termos do Docker

Vamos revisar alguns termos importantes:

- Um **Dockerfile** define o conteúdo de um container;
- A **imagem Docker** é a base para construir um container;
- E o **container Docker** é uma instância de execução baseada na imagem.

Instruções vs. comandos Docker

Instruções Docker são usadas nos Dockerfiles para construir imagens. Enquanto que **Comandos Docker** são executados pela interface de linha de comando (CLI) para gerenciar objetos Docker.

Formato de um Dockerfile

O Dockerfile contém instruções que começam com um comentário, seguidas pela instrução em letras maiúsculas e seu argumento em minúsculas. A ordem das instruções é importante e define a sequência de construção.

Instruções Docker

As instruções em um Dockerfile são executadas na ordem em que aparecem. Ele geralmente começa com metadados, comentários, argumentos ou a instrução FROM.

As principais instruções do Docker são:

- **FROM**: define a imagem base a ser usada.
- **COPY**: copia arquivos ou pastas para o container.
- **RUN**: executa comandos dentro do container.
- **CMD**: especifica o comando padrão que será executado quando o container iniciar.

As instruções são montadas começando pela **FROM**, seguida de **COPY**, **RUN**, e finalizando com **CMD**.

Comandos Docker

Os principais comandos Docker são:

- **docker build**: cria uma imagem a partir de um Dockerfile.
- **docker run**: cria e executa um container a partir de uma imagem Docker.

Para explorar mais comandos Docker e entender o gerenciamento de containers e imagens, vamos abordar os principais comandos para operações cotidianas.

Comandos básicos para gerenciamento de imagens e containers:

- `docker ps`: lista os containers em execução no momento, ou seja apenas containers ativos.
- `docker ps -a`: lista todos os containers, incluindo os que estão parados.
- `docker rm`: remove um ou mais containers parados. Para remover múltiplos containers, forneça os IDs ou nomes separados por espaço.
- `docker rmi`: remove uma ou mais imagens Docker. Para remover uma imagem, forneça o ID ou nome da imagem;
- `docker logs`: exibe os logs gerados por um container em execução ou que já foi executado, esse comando pode ser útil para depurar erros ou ver a saída da aplicação ;
- `docker exec`: executa um comando dentro de um container em execução. Comumente usado para acessar o terminal de um container com bash ou sh. Veja alguns exemplos de execução:

```
```bash

dockerexec -it <container_id>bash # abre um terminal bash
interativo dentro do container

dockerexec<container_id><command> # executa um comando
específico no container

```
```

Esses comandos são fundamentais para gerenciar imagens e containers no Docker. Eles facilitam a automação e o gerenciamento de ambientes, tornando o desenvolvimento e a operação mais eficientes. Para mais informações e casos de uso, você pode acessar a documentação do [Docker](#).

Storage no Docker

Agora vamos explorar o conceito de volumes em Docker. Volumes são componentes fundamentais para o gerenciamento de dados persistentes em containers do Docker. Eles permitem que os dados sejam armazenados fora do ciclo

de vida de um container, garantindo que informações importantes não sejam perdidas quando um container é reiniciado ou deletado.

Volumes são áreas de armazenamento gerenciadas pelo Docker e que são totalmente independentes do sistema operacional subjacente, o que os torna portáteis e seguros para diferentes plataformas.

Tipos de mounts no Docker

O volume do Docker é um tipo de mount. Mount é o processo de conectar uma parte do sistema de arquivos do host ou uma área de armazenamento ao sistema de arquivos do container, permitindo que o container leia ou escreva dados nesse local. É como criar uma "ponte" entre o ambiente do host e o container, possibilitando o compartilhamento de dados entre eles. Esse tipo de recurso é necessário, pois os containers são isolados por padrão.

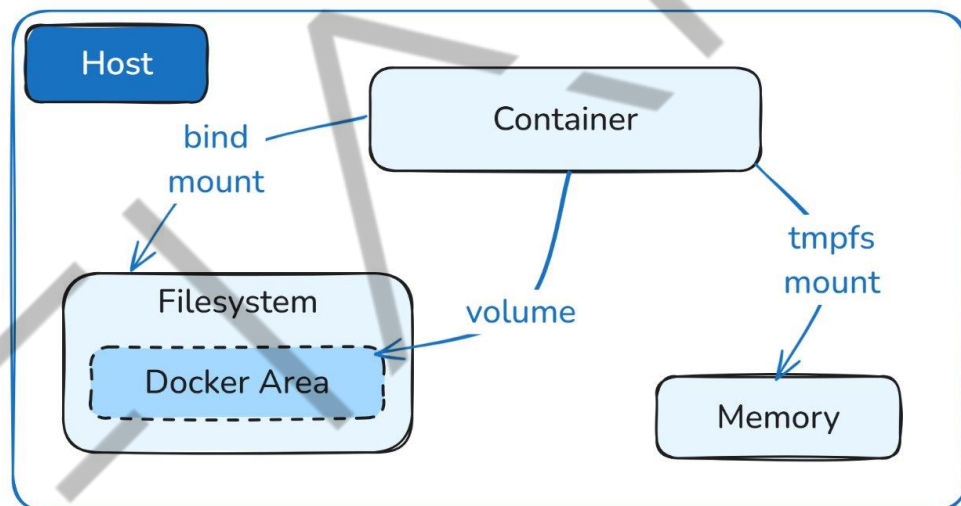


Figura 4 - Tipos de storage do Docker

Fonte: Adaptado de [Docker](#) (2025)

A Figura acima expressa os três tipos principais de mounts que podem ser utilizados com containers Docker: **bind mount**, **volume** e **tmpfs mount**. (Nessa aula não nos aprofundaremos nos tipos bindmount e tmpfsmount).

Mas, basicamente, o bind mount usa diretórios/arquivos específicos do host e não é gerenciado pelo Docker (o Docker não mantém controle ou restrições sobre o diretório montado), e esse tipo de mount é útil para compartilhamento de arquivos específicos entre o host e o container.

O tmpfs mount usa memória (RAM) para armazenamento temporário e os dados são perdidos quando o container é parado, é ideal para os casos de utilização de dados sensíveis ou temporários que não precisam ser persistentes (como arquivos de cache).

Já o volume é gerenciado pelo Docker e armazenado na área dedicada do Docker no sistema de arquivos do host. Na figura, o volume é representado no desenho pela linha "volume", que conecta a área gerenciada pelo Docker (Docker Area) ao container.

As características do volume Docker são:

- É persistente e permanece mesmo se o container for excluído;
- É o método recomendado para armazenamento de dados persistentes, como bancos de dados;
- Como já mencionado, é gerenciado inteiramente pelo Docker, com mais controle e isolamento do que bind mounts.

Exemplo de persistência de dados no volume do Docker

Para criar um volume nomeado, execute:

```
```bash
docker volume create dados_app
```
```

Agora, executamos um container usando este volume e criamos um arquivo no volume.

```
```bash
docker run -d --name app_volumes -v dados_app:/app python:3.9-slim \
bash -c 'echo "Hello, Docker!" >> /app/mensagem.txt'
```
```

Agora, podemos verificar se o arquivo foi persistido no volume. A partir daqui vamos parar e remover o container.

```
```bash

docker stop app_volumes

docker rm app_volumes

```
```

Vamos recriar o container usando o mesmo volume.

```
```bash

docker run -it --name app_volumes -v dados_app:/app
python:3.9-slim bash

```
```

E, ao acessar o container, é esperado que o arquivo mensagem.txt esteja no volume.

```
```bash

cat /app/mensagem.txt

```
```

Se o seguinte conteúdo do arquivo for exibido, o volume está funcionando corretamente e o dado foi persistido!

```
```

Hello, Docker!

```
```

Docker compose

O Docker Compose é uma ferramenta que permite definir e gerenciar aplicações multi-container. Ele utiliza um arquivo YAML para configurar todos os serviços, redes e volumes necessários para uma aplicação e permite que você

possa subir, parar e gerenciar todos eles com um único comando. Imagine o Docker Compose como um “orquestrador” que gerencia e interliga diversos containers, facilitando o gerenciamento de aplicações complexas.

Com Docker Compose você pode definir:

- **Serviços:** cada serviço corresponde a um container Docker com suas especificações (imagem, variáveis de ambiente, volumes, portas etc).
- **Redes:** as redes que interligam os containers, facilitando a comunicação entre eles.
- **Volumes:** os volumes que persistem dados entre os containers e o host.

O Docker Compose simplifica a criação e o gerenciamento de aplicações multi-container. Ao invés de executar múltiplos comandos `docker run`, você define sua aplicação em um arquivo `docker-compose.yml` e utiliza comandos do Docker Compose para subir, gerenciar e parar toda a aplicação. Além disso, Docker Compose facilita o trabalho em equipe e a garantia de que todos os containers serão executados com as mesmas configurações, eliminando surpresas e facilitando o processo de desenvolvimento.

Arquivo `docker-compose.yml`

O arquivo `docker-compose.yml` é onde você declara todos os serviços que compõem sua aplicação. Veja um exemplo de um arquivo `docker-compose.yml`:

```
```yaml
version: "3.9"

services:

 app1:

 image: python:3.9-slim

 volumes:

- ./app1:/app

 command: "python /app/app_1.py"
```

```
app2:
 image: python:3.9-slim
 volumes:
- ./app2:/app
 command: "python /app/app_2.py"
...
```

Nesse exemplo, temos:

- **version:** a versão do formato do `docker-compose.yml`.
- **services:** contém a definição de todos os serviços.
- **app1:** um serviço chamado `app1` que usa a imagem `python:3.9-slim`, monta o diretório local `./app1` no diretório `/app` do container e executa o comando `python /app/app_1.py`.
- **app2:** um serviço chamado `app2` que usa a imagem `python:3.9-slim`, monta o diretório local `./app2` no diretório `/app` do container e executa o comando `python /app/app_2.py`.

### Comandos do Docker Compose

Aqui estão alguns comandos essenciais do Docker Compose:

- **docker-compose up:** cria e inicia todos os containers definidos no arquivo `docker-compose.yml`. Por padrão, ele procura o arquivo no diretório atual.
- **docker-compose up -d:** executa o comando `up` em modo detached, ou seja, ele executa os containers em background, sem travar seu terminal.
- **docker-compose down:** remove todos os containers, redes e volumes criados pelo `docker-compose`.

- `docker-compose stop`: para todos os containers, mas os mantém no sistema.
- `docker-compose start`: inicia os containers parados.
- `docker-compose logs`: visualiza os logs dos containers.

### Exercício 1

Vamos criar um container Docker que executa um script Python imprimindo "Hello world!". É necessário ter o Docker Desktop instalado em sua máquina local.

1. Crie um arquivo Python chamado `app.py` em uma pasta chamada `exercicio_1` com o seguinte conteúdo:

```
```python
print("Hello world!")
```
```

2. Crie um arquivo `Dockerfile` no mesmo diretório com o seguinte conteúdo:

```
```Dockerfile
# Use a imagem base do Python
FROM python:3.9-slim

# Essa instrução do Docker copia o script Python para o
container

COPY app.py /app.py

# Essa instrução define o comando para rodar o script quando o
container Docker for executado
```

```
CMD ["python", "/app.py"]
```

```
...
```

3. Construa (ou “faça o build”) a imagem Docker com nome “hello-world” executando o comando no terminal:

```
``bash
```

```
docker build -t hello-world .
```

```
...
```

É esperado que a imagem criada esteja visível no Docker Desktop, conforme a figura a seguir:

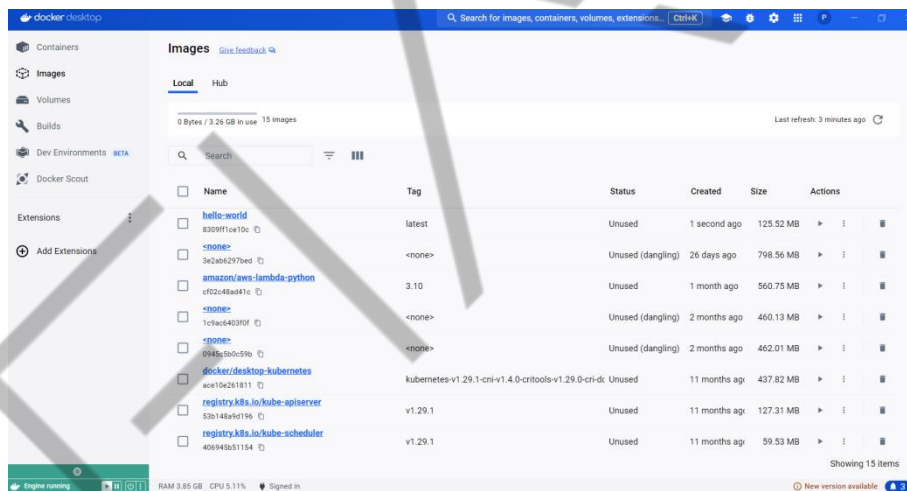


Figura 5 - Print do Docker Desktop com a imagem **hello-world** criada
Fonte: Elaborado pelo autor (2025)

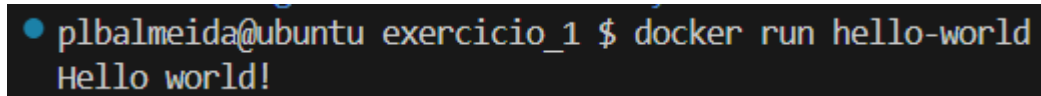
4. Execute o container com o comando:

```
``bash
```

```
docker run hello-world
```

```
...
```

Com isso, é esperado executar um container que imprime "Hello world!", conforme a figura a seguir:



```
plbalmeida@ubuntu exercicio_1 $ docker run hello-world
Hello world!
```

Figura 6 - Print do terminal com a execução do container `hello-world`
Fonte: Elaborado pelo autor (2025)

Exercício 2

Agora vamos criar uma pasta chamada `exercicio_2` copiar e modificar o `app.py` para incluir uma biblioteca externa do Python (`requests`) e atualizar o `Dockerfile` com instruções adicionais, como `RUN` para instalar dependências, além de incluir `ENV`, `WORKDIR` e `EXPOSE`.

1. Modificando o `app.py`

Atualize o conteúdo do arquivo `app.py` para fazer uma requisição HTTP simples usando a biblioteca `requests`. O novo conteúdo do `app.py` será:

```
python
import requests

response = requests.get("https://api.github.com")
print(f"Status Code: {response.status_code}")
print(f"Response JSON: {response.json()}")
...
```

2. Atualizando o `Dockerfile`

O novo `Dockerfile` incluirá instruções para:

- Instalar o `requests` via `pip`;

- Definir uma variável de ambiente (**ENV**);
- Configurar um diretório de trabalho (**WORKDIR**);
- Expor uma porta (**EXPOSE**), caso a aplicação seja uma API ou serviço.

Segue o novo conteúdo do **Dockerfile**:

```
``Dockerfile

# Use a imagem base do Python
FROM python:3.9-slim

# Configura uma variável de ambiente
ENV PYTHONUNBUFFERED=1

# Define o diretório de trabalho no container
WORKDIR /app

# Copia o arquivo de script Python para o container
COPY app.py /app/app.py

# Instala a biblioteca requests
RUN pipinstallrequests

# Expõe a porta 80 (caso a aplicação seja uma API, por exemplo)
EXPOSE 80
```

```
# Define o comando para rodar o script quando o container for executado

CMD ["python", "app.py"]

...
```

Explicação das instruções adicionadas no **Dockerfile**:

- **ENV PYTHONUNBUFFERED=1**: define uma variável de ambiente para evitar o buffer na saída do Python, garantindo que os logs apareçam em tempo real.
- **WORKDIR /app**: configura o diretório de trabalho dentro do container. Isso significa que todos os comandos subsequentes no Dockerfile serão executados a partir de **/app**.
- **RUN pip install requests**: instala a biblioteca requests no container.
- **EXPOSE 80**: informa que a aplicação está configurada para escutar na porta 80. Embora não seja necessário para a execução do script atual, essa instrução pode ser útil para APIs ou serviços web.

3. Construindo a imagem

Para construir a imagem Docker com o nome **hello-requests**, execute o seguinte comando no terminal:

```
```bash

docker build -t hello-requests .

...
```

### 4. Executando o container

Agora, execute o container para visualizar a saída do script:

```
```bash

docker run hello-requests

```
```

É esperado que seja mostrado o status e a resposta da requisição HTTP feita para a API do GitHub:



```
plbalmeida@ubuntu: exercicio 2 $ docker run hello-requests
Status Code: 200
Response JSON: {'current_user_url': 'https://api.github.com/user', 'current_user_authorizations_html_url': 'https://github.com/settings/connections/applications/{client_id}', 'authorizations_url': 'https://api.github.com/authorizations', 'code_search_url': 'https://api.github.com/search/code?q={query}&page={page}&per_page={per_page}&sort={sort}&order={order}', 'commit_search_url': 'https://api.github.com/search/commits?q={query}&page={page}&per_page={per_page}&sort={sort}&order={order}', 'emails_url': 'https://api.github.com/user/emails', 'emojis_url': 'https://api.github.com/emojis', 'events_url': 'https://api.github.com/events', 'feeds_url': 'https://api.github.com/feeds', 'followers_url': 'https://api.github.com/user/followers', 'following_url': 'https://api.github.com/user/following/{target}', 'gists_url': 'https://api.github.com/gists/{gist_id}', 'hub_url': 'https://api.github.com/hub', 'issue_search_url': 'https://api.github.com/search/issues?q={query}&page={page}&per_page={per_page}&sort={sort}&order={order}', 'issues_url': 'https://api.github.com/issues', 'keys_url': 'https://api.github.com/user/keys', 'label_search_url': 'https://api.github.com/search/labels?q={query}&repository_id={repository_id}&page={page}&per_page={per_page}', 'notifications_url': 'https://api.github.com/notifications', 'organization_url': 'https://api.github.com/orgs/{org}', 'organization_repositories_url': 'https://api.github.com/orgs/{org}/repos?type={type}&page={page}&per_page={per_page}', 'organization_teams_url': 'https://api.github.com/orgs/{org}/teams', 'public_gists_url': 'https://api.github.com/gists/public', 'rate_limit_url': 'https://api.github.com/rate_limit', 'repository_url': 'https://api.github.com/repos/{owner}/{repo}', 'repository_search_url': 'https://api.github.com/search/repositories?q={query}&page={page}&per_page={per_page}&sort={sort}&order={order}', 'current_user_repositories_url': 'https://api.github.com/user/repos?type={type}&page={page}&per_page={per_page}&sort={sort}', 'starred_url': 'https://api.github.com/user/starred/{owner}/{repo}', 'starred_gists_url': 'https://api.github.com/gists/starred', 'topic_search_url': 'https://api.github.com/search/topics?q={query}&page={page}&per_page={per_page}', 'user_url': 'https://api.github.com/users/{user}', 'user_organizations_url': 'https://api.github.com/user/orgs', 'user_repositories_url': 'https://api.github.com/users/{user}/repos?type={type}&page={page}&per_page={per_page}', 'user_search_url': 'https://api.github.com/search/users?q={query}&page={page}&per_page={per_page}&sort={sort}&order={order}'}
```

Figura 7 - Print do terminal com a execução do container `hello-requests` e retorno com status 200

Fonte: Elaborado pelo autor (2025)

### Exercício 3

Agora vamos utilizar um volume Docker para criar um banco de dados SQL persistente usando a imagem oficial do MySQL, incluindo a inserção de registros e a execução de queries SQL para checar o retorno.

Crie um volume nomeado para armazenar os dados do banco.

```
```bash

docker volume create dados_mysql

```
```

Inicie um container MySQL utilizando o volume criado.

```
```bash

docker run -d --name mysql_container \

-e MYSQL_ROOT_PASSWORD=senha_secreta \
```

```
-e MYSQL_DATABASE=meu_banco \  
-v dados_mysql:/var/lib/mysql \  
-p 3306:3306 \  
mysql:8.0  
...
```

Explicação:

- `MYSQL_ROOT_PASSWORD`: define a senha para o usuário root.
- `MYSQL_DATABASE`: cria automaticamente um banco de dados chamado `meu_banco`.
- `-v dados_mysql:/var/lib/mysql`: conecta o volume ao diretório de dados do MySQL.
- `-p 3306:3306`: expõe a porta do banco de dados para acesso local.

Vamos inserir registros no banco de dados. Para isso, acesse o container:

```
```bash  
docker exec -it mysql_container bash
...
```

Abra o cliente MySQL:

```
```bash  
mysql -uroot -p  
...
```

Conecte-se ao banco de dados `meu_banco`:

```
```mysql
USE meu_banco;
```
```

Crie uma tabela chamada **usuarios**:

```
```mysql
CREATE TABLE usuarios (
 id INT AUTO_INCREMENT PRIMARY KEY,
 nome VARCHAR(50) NOT NULL,
 email VARCHAR(50) NOT NULL
);
```
```

Insira registros na tabela:

```
```mysql
INSERT INTO usuarios (nome, email) VALUES ('Alice',
'alice@email.com');

INSERT INTO usuarios (nome, email) VALUES ('Bob',
'bob@email.com');

INSERT INTO usuarios (nome, email) VALUES ('Carlos',
'carlos@email.com');
```
```

Cheque se os registros foram inseridos:

```
```mysql  

SELECT * FROM usuarios;

```
```

O resultado esperado:

```
mysql> SELECT * FROM usuarios;  
+----+-----+-----+  
| id | nome  | email          |  
+----+-----+-----+  
1	Alice	alice@email.com
2	Bob	bob@email.com
3	Carlos	carlos@email.com
+----+-----+-----+
```

Figura 8 – Resultado
Fonte: Elaborado pelo autor (2025)

Agora vamos testar a persistência dos dados, uma propriedade de volume do Docker.

Pare e remova o container:

```
```bash  

docker stop mysql_container

docker rm mysql_container

```
```

Recrie o container utilizando o mesmo volume:

```
```bash  

docker run -d --name mysql_container \
-e MYSQL_ROOT_PASSWORD=senha_secreta \
-v dados_mysql:/var/lib/mysql \
mysql
```

```
-p 3306:3306 \
mysql:8.0
...
```

Verifique se os registros ainda existem. Acesse o container e o cliente MySQL:

```
```bash  
docker exec -it mysql_container bash  
...
```

E depois:

```
```mysql  
mysql -uroot -p
...
```

Conecte-se ao banco `meu_banco` e consulte a tabela `usuarios`:

```
```mysql  
USE meu_banco;  
SELECT * FROM usuarios;  
...
```

O resultado deve exibir os registros previamente inseridos:

```
mysql> SELECT * FROM usuarios;
+----+-----+-----+
| id | nome  | email                |
+----+-----+-----+
| 1  | Alice | alice@email.com      |
| 2  | Bob   | bob@email.com        |
| 3  | Carlos| carlos@email.com     |
+----+-----+-----+
```

Figura 9 – Resultado (2)
Fonte: Elaborado pelo autor (2025)

Exercício 4

Vamos criar um projeto simples que utiliza dois scripts Python (`app_1.py` e `app_2.py`) e um `docker-compose.yml` para orquestrar a execução de ambos. Primeiro, vamos criar a seguinte estrutura de diretórios:

```
...
exercicio_4/
├── app1/
│   ├── app_1.py
├── app2/
│   ├── app_2.py
└── docker-compose.yml
...
```

Crie o `app_1.py`

No diretório `app1/`, crie o arquivo `app_1.py` com o seguinte conteúdo:

```
```python  
print("Hello world!")
```
```

Crie o `app_2.py`

No diretório `app2/`, crie o arquivo `app_2.py` com o seguinte conteúdo:

```
```python  
print("Hello world again!")
```
```

Crie o `docker-compose.yml`

Na raiz do diretório `exercicio_4/`, crie o arquivo `docker-compose.yml` com o seguinte conteúdo:

```
```yaml  
version: "3.9"
services:
 app1:
 image: python:3.9-slim
volumes:
- ./app_1:/app
working_dir: /app # Define /app como o diretório de trabalho
command: "python app_1.py"
 app2:
 image: python:3.9-slim
```

```
volumes:
- ./app_2:/app
working_dir: /app # Define /app como o diretório de trabalho
command: "python app_2.py"
...
```

Inicie a aplicação com Docker Compose

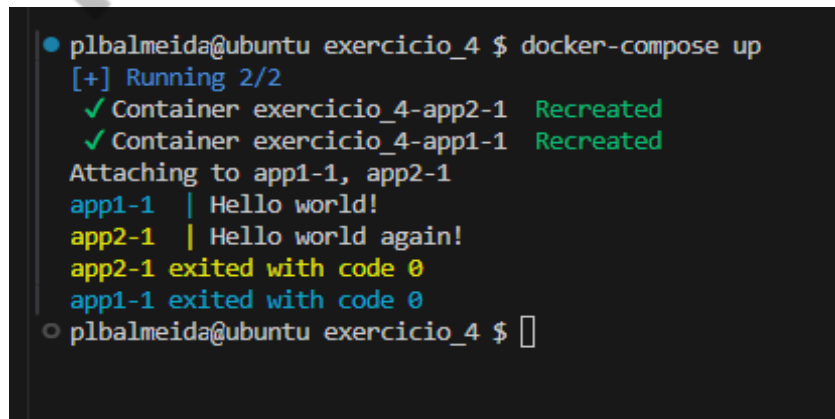
Na raiz do diretório `exercicio_4/`, execute o comando:

```
```bash  
docker-composeup  
...
```

O Docker Compose fará o download das imagens e executará os containers. Os outputs serão exibidos no seu terminal.

Verifique os logs

Verifique as saídas no seu terminal para ver os logs gerados por cada aplicação. Você deve visualizar "Hello world!" sendo impresso a partir de um container e "Hello world again!" sendo impresso a partir do outro, conforme a figura a seguir:

A terminal window with a dark background and light-colored text. It shows the command 'docker-compose up' being executed. The output indicates that two containers, 'exercicio_4-app2-1' and 'exercicio_4-app1-1', were recreated. It then shows the logs for both containers: 'app1-1 | Hello world!' and 'app2-1 | Hello world again!'. Both containers exited with code 0. The prompt shows the user is at the 'exercicio_4' directory.

```
● plbalmeida@ubuntu exercicio_4 $ docker-compose up  
[+] Running 2/2  
✓ Container exercicio_4-app2-1 Recreated  
✓ Container exercicio_4-app1-1 Recreated  
Attaching to app1-1, app2-1  
app1-1 | Hello world!  
app2-1 | Hello world again!  
app2-1 exited with code 0  
app1-1 exited with code 0  
○ plbalmeida@ubuntu exercicio_4 $
```

Figura 10 – Logs

Fonte: Elaborado pelo autor (2025)

Agora, pare a aplicação.

Na raiz do diretório `exercicio_4/`, execute o comando:

```
```bash  
docker-compose down
```
```

O Docker Compose removerá todos os containers criados para a aplicação.

Este exercício demonstrou como o Docker Compose facilita a execução de múltiplas aplicações simultaneamente, permitindo a criação de ambientes complexos de forma simples e organizada. E como vimos, o arquivo `docker-compose.yml` torna o processo de deploy muito mais gerenciável.

Networking no Docker

Nesta aula, o foco principal foi colaborar com estudantes na compreensão e aplicação prática do uso de Docker para containerização, para que possamos entender como usar essa ferramenta para colocar modelos de ML em produção de forma prática e eficiente.

Embora o tema de **networking** seja importante para a administração e orquestração de aplicações mais complexas, ela extrapola o objetivo da disciplina e poderia desviar a atenção do que realmente importa.

O Docker Networking facilita a criação de redes customizadas para containers, garantindo isolamento, segurança e comunicação direta entre serviços, como um servidor de aplicação que interage com um banco de dados por exemplo.

MERCADO, CASES E TENDÊNCIAS

Como material complementar sobre o tema de network, sugerimos os seguintes links da documentação oficial da Docker:

- Visão geral de redes no [Docker](#).
- Trabalhando com redes no [Docker](#).

Esses materiais oferecem uma base sólida para quem deseja explorar o uso de Docker em arquiteturas mais complexas, complementando os conhecimentos adquiridos no nosso curso.

O [artigo](#) da Docker fala sobre um caso de uso da combinação do Docker com a plataforma Kaskada – que é uma plataforma de processamento de dados em tempo real. O material mostra como essa stack facilita a reprodutibilidade completa de projetos de ML/IA, além de explicar como usar containers Docker para encapsular todas as partes de um pipeline de ML/IA, incluindo código, dependências, dados e infraestrutura, garantindo que o mesmo ambiente possa ser recriado facilmente em diferentes fases do desenvolvimento e em diversas máquinas.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, vimos os conceitos fundamentais de containerização com Docker, abordando as diferenças entre virtualização tradicional com VMs, e virtualização em nível de sistema operacional (OS-level), que caracteriza os containers Docker.

Exploramos como o Docker utiliza essa tecnologia para criar ambientes isolados e leves, otimizando o uso de recursos e facilitando a portabilidade de aplicações. Também aprendemos sobre a criação de Dockerfiles, construção de imagens e execução de containers, como utilizar registros Docker para armazenar e compartilhar imagens, além de travarmos contato com volumes do Docker. Ao final, aplicamos esses conceitos em exercícios práticos para construir e executar containers para diferentes finalidades.

REFERÊNCIAS

DEV MEDIA. **Virtualização de servidores**. DevMedia, [s.d.]. Disponível em: <<https://www.devmedia.com.br/virtualizacao-de-servidores/30820>>. Acesso em: 17jan.. 2025.

DOCKER. **Docker overview**. Docker, [s.d.]. Disponível em: <<https://docs.docker.com/get-started/docker-overview/>>. Acesso em: 17 jan.. 2025.

DOCKER. **Getting started.**, [s.d.] Disponível em: <<https://docs.docker.com/compose/gettingstarted/>>. Acesso em: 17 jan. 2025.

DOCKER. **Volumes**. Docker Documentation, [s.d.]. Disponível em: <<https://docs.docker.com/engine/storage/volumes/>>. Acesso em: 17 jan. 2025.

DOCKER. **What is a container?**. Docker, [s.d.]. Disponível em: <<https://www.docker.com/resources/what-container/>>. Acesso em: 17 jan. 2025.

RED HAT. **Containers vs. VMs: qual é a diferença?**. Red Hat, 2024. Disponível em: <<https://www.redhat.com/pt-br/topics/containers/containers-vs-vm>>. Acesso em: 17 jan. 2025.

GLOSSÁRIO

CI/CD: abreviação de Continuous Integration/Continuous Delivery ou Continuous Deployment (Integração Contínua e Entrega/Implantação Contínua).

Onde:

- **CI:** integração contínua envolve a automação da construção e teste de código toda vez que mudanças são feitas.
- **CD:** entrega/implantação contínua automatiza a entrega do software para ambientes de produção, garantindo que ele esteja sempre pronto para ser implantado.

DevOps: conjunto de práticas, ferramentas e uma cultura que integra as equipes de desenvolvimento (Dev) e operações (Ops) para automatizar e melhorar continuamente os processos de entrega e infraestrutura de software. O objetivo é acelerar a entrega de software com alta qualidade e confiabilidade.

Entrega Contínua: prática de desenvolvimento de software onde as alterações de código são automaticamente testadas e prontas para serem implantadas em produção a qualquer momento. A entrega contínua exige uma forte automação de testes e integração de processos para garantir qualidade e rapidez.

Inteligência Artificial (IA): campo da ciência da computação que se dedica à criação de sistemas capazes de realizar tarefas que normalmente requerem inteligência humana, como reconhecimento de fala, visão computacional, raciocínio e aprendizado.

Kernel: o núcleo de um sistema operacional. Ele gerencia os recursos de hardware do computador e atua como intermediário entre o software e o hardware. É responsável por tarefas como gerenciamento de memória, processos e dispositivos de entrada/saída.

Machine Learning: subcampo da Inteligência Artificial que utiliza algoritmos e modelos matemáticos para ensinar máquinas a aprenderem padrões a partir de dados e fazer previsões ou tomar decisões sem serem explicitamente programadas para isso.

Microserviços: padrão de arquitetura de software que fragmenta uma aplicação em pequenos serviços independentes, cada um executando uma

funcionalidade específica. Esses serviços podem ser desenvolvidos, implantados e escalados de forma independente, facilitando a manutenção e evolução do sistema.

Sistema Host: o sistema operacional ou ambiente no qual outro sistema operacional (chamado de sistema guest) ou aplicações específicas estão sendo executados. Por exemplo, em uma máquina virtual, o sistema host hospeda o software de virtualização.

EMANIP

PALAVRAS-CHAVE

Containerização. Docker. Engenharia de Software.

EMSE

The background is a dark blue field filled with numerous small, light blue dots, resembling a starry sky. Overlaid on this are several large, wavy, translucent lines in shades of blue, teal, and yellow. These lines flow from the left side towards the right, creating a sense of motion. Scattered throughout the composition are various geometric shapes: a thin vertical line, a circle containing the number '7', a small circle, an 'X' mark, a small circle, and a hexagon in the bottom right corner.

POSTECH